

Enunciado-Final-PLP-18-02-25

Sumario

- 1 Haskell
- 2 Induccion Estructural
- 3 Deduccion Natural
- 4 Prolog

Haskell

```
data Form = Prop String
          | And Form Form
          | Or Form Form
          | Neg Form
```

(A) Definir y dar el tipo de `foldForm`

(B) Definir `fnn :: Form -> Bool -> Form` usando `foldForm`. Que pasa una formula `x` a forma normal negada si el booleano es `True` y pasa a la negacion de `x` a forma normal negada si el booleano es `False`.

Ej:

```
fnn (And (Prop "x") Neg(Or (Prop "y") Neg(Prop "z")))) True = (And (Prop "x") (And Neg(Prop "y")
(Prop "z")))
```

```
fnn (And (Prop "x") Neg(Or (Prop "y") Neg(Prop "z")))) False = (Or Neg(Prop "x") (Or (Prop "y")
Neg(Prop "z")))
```

Induccion Estructural

Dados

```
{A0} alt f g [] = []
{A1} alt f g (x:xs) = f x : alt g f xs
```

Demostrar `alt g1 g2 . alt f1 f2 = alt (g1 . f1) (g2 . f2)`

Deduccion Natural

Dadas las nuevas reglas :

$$\frac{\Gamma, \Delta, \neg \tau \vdash \tau}{\Gamma \vdash \Delta \tau} \quad \Delta i$$

$$\frac{\Gamma \vdash \Delta \tau \quad \Gamma \vdash \Delta \neg \tau}{\Gamma \vdash \tau} \Delta_e$$

Demostrar las siguientes formulas:

- a) $\vdash \tau \Rightarrow \Delta \tau$
- b) $\Delta \tau \vdash \Delta \neg \neg \tau$ (ojo, no se puede que $\neg \neg \neg \tau = \neg \tau$)
- c) $\Delta \tau, \Delta \neg \tau \vdash \sigma$ sugiere usar $\perp_e$ y el ítem anterior.

Prolog

En prolog definir λ -terminos como

- `var(X)` donde X es un numero natural. Representa al uso de la variable numero X.
- `lam(X, M)` donde X es un numero natural y M es un λ -termino. Representa la ligadura de la variable numero X con respecto al λ -termino M.
- `app(M, N)` donde M y N son λ -terminos. Representa a la aplicacion.

Ej:

`lam(1, lam(2, lam(3, app(var(1), app(var(2), var(3)))))` es equivalente al termino $\lambda f. \lambda g. \lambda x. f(g\ x)$

(A) Definir `variablesLibres(+M, -L)` que instancia en una lista L las variables libres del termino M.

Ej: `variablesLibres(lam(1, app(var(1), var(2))), L)` instancia `L = [2]`.

(B) Definir `tamano(+M, -T)` que calcula el tamaño de un termino. `var(X)` suma 1. `lam(X, M)` suma `1 + tamano(M)`. `app(M, N)` suma `1 + tamano(M) + tamano(N)`.

Ej: `tamano(app(var(1), lam(1, var(2))))` instancia `T = 4`.

(C) Definir `generarLambdaTerminos(+xs, -M)` que dada una lista de numeros naturales XS instancia en M λ -terminos infinitos. Sugerencia: Crear los λ -terminos en base al tamaño.

Obtenido de «<https://www.cubawiki.com.ar/index.php?title=Enunciado-Final-PLP-18-02-25&oldid=17738>»